Automate FW testing with MDBCore scripting, part of the MPLAB®X ecosystem (module3b: advanced test automation with mdbcs)

Stefan Luethin

December '23

Agenda

1: what is the ***MPLABX*** ecosystem ***MDBCore***, ***SDK*** -> how to use for automated FW testing

2: compile without IDE

3: Automated *FW-testing* with different ***MDBCore*** use-models

   a. simple ***mdb*** -> DOS/bash-script, easy to use but has limits
   b. more advanced ***mdbcs*** -> *.jar module needs scripting-language with Java-interface, more complex but solves ***mdb*** limits and opens door into Java-universe
   c. full, but most complex usage of ***MDBCore*** with Java -> SDK

4: summary

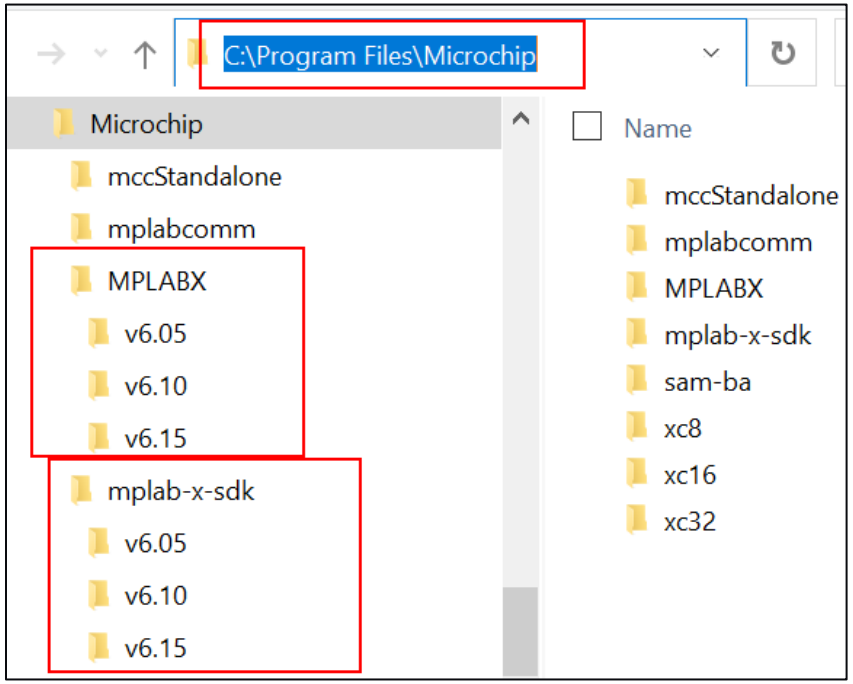
M3b/mdbcs: preparation to use *mdbcs*

To use mdbcs

- First what tools, programs are needed ?
- Second how to configure your environment ?
- Third, need to build mdbcs -> SDK has instructions

M3b/mdbcs: tools to use *mdbcs*

First, tools and programs needed to use **mdbcs** ?



- get + install SDK (-> www.opensource4pic.org - just a zip-file, recommended to install parallel to MPLABX, -> eg install into: C:\Program Files\Microchip\mplab-x-sdk\v6.15)
- get + install NB (likely any version is ok, but tested with https://netbeans.apache.org/download/nb18/ -> eg install into: C:\Program Files\NetBeans-18)
- get + install groovy (likely any version is ok, but tested with groovy-v4.0.12 from groovy.apache.org -> eg install into: C:\Program Files\Groovy\v4.0.12)
- get + install python-v2.x (tested with v2.7 from www.python.org/downloads/release/python-2716/ -> eg install into: C:\Program Files\Python\v2.7.14) (!MPLABX and MDBCore need Java-IF which is available through jython which is frozen on python-v2.7x!!)
- get + install MPLABX + XC32 -> eg install into: C:\Program Files\Microchip\MPLABX\v6.15 , C:\Program Files\Microchip\xc32\v4.35)

M3b/mdbcs: configure environment to use *mdbcs*

Second, what do you need to configure to use **mdbcs** ?

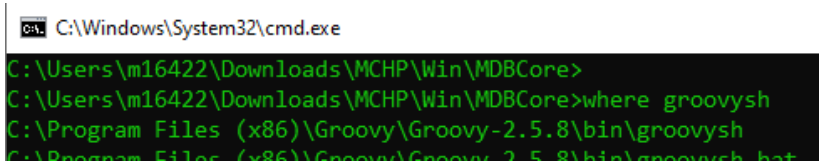
- Configure the OS-searchpath %PATH%
  - MPLABX_HOME = C:\Program Files\Microchip\MPLABX\v6.15
  - JAVA_HOME = %MPLABX_HOME%\sys\java\zulu8.64.0.19-ca-fx-jre8.0.345-win_x64
  - GNU_HOME = %MPLABX_HOME%\gnuBins\GnuWin32
  - GROOVY_HOME = C:\Program Files\Groovy\v4.0.12
  - PYTHON2_HOME = C:\Program Files\Python\v2.7.14 (jython-project frozen with python-v2.7)
  - set PATH=%GROOVY_HOME%\bin;%JAVA_HOME%\bin; %GNU_HOME%\bin;%PYTHON2_HOME%

- CLASSPATH, what is it and why needed?
  - Given a Java-module `myJavaModule` with a collection of Java-functions and -classes
  - It is pre- compiled into `myJavaModule.jar`
  - To use this Java-module in another Java-project, you include it with `import myJavaModule`
  - But where should the Java interpreter look for this `myJavaModule.jar` on your computer?
  - This information comes from so called $CLASSPATH which is like the OS-searchpath just a list of paths where to look for `myJavaModule.jar` if `import myJavaModule` is found (later slide)

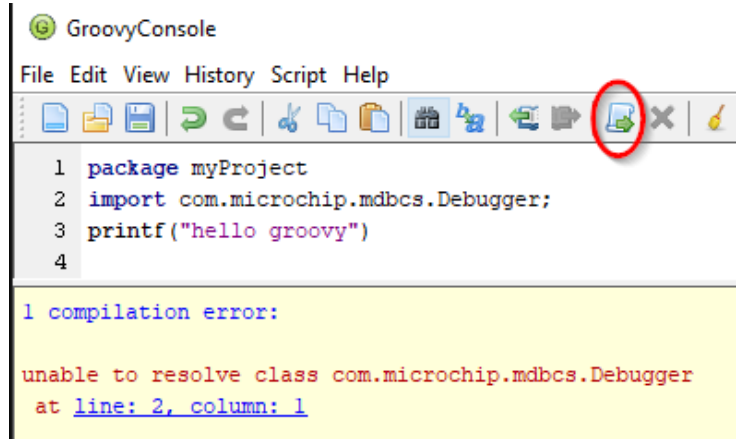
M3b/mdbcs: configure environment to use *mdbcs*

...continue 2.step configuring environment: test environment

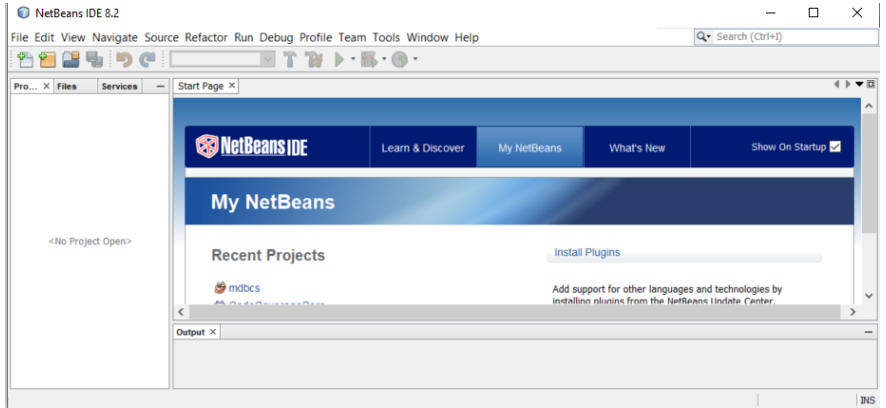
- OS-searchpath
  - Java version of MPLABX is used from <mplabx>\sys\java\zulu*\jre\bin
    ->see later slide on 'Building mdbcs' and test with (should point to above MPLABX-version you target)
    `(dos)> where java` #-> should find java.exe in MPLABX/sys/java...
    `(dos)> echo %Path%` #-> all %-variables resolved?
- Groovy
  - Add Groovy to searchpath `(dos:)> where groovy`



  - Start groovyconsole, enter 3 simple lines of groovy left
  - Try to execute, BUT it'll fail with this error as your CLASSPATH (=java-searchpath) is not set, so mdbcs.jar not found -> see later how to fix this



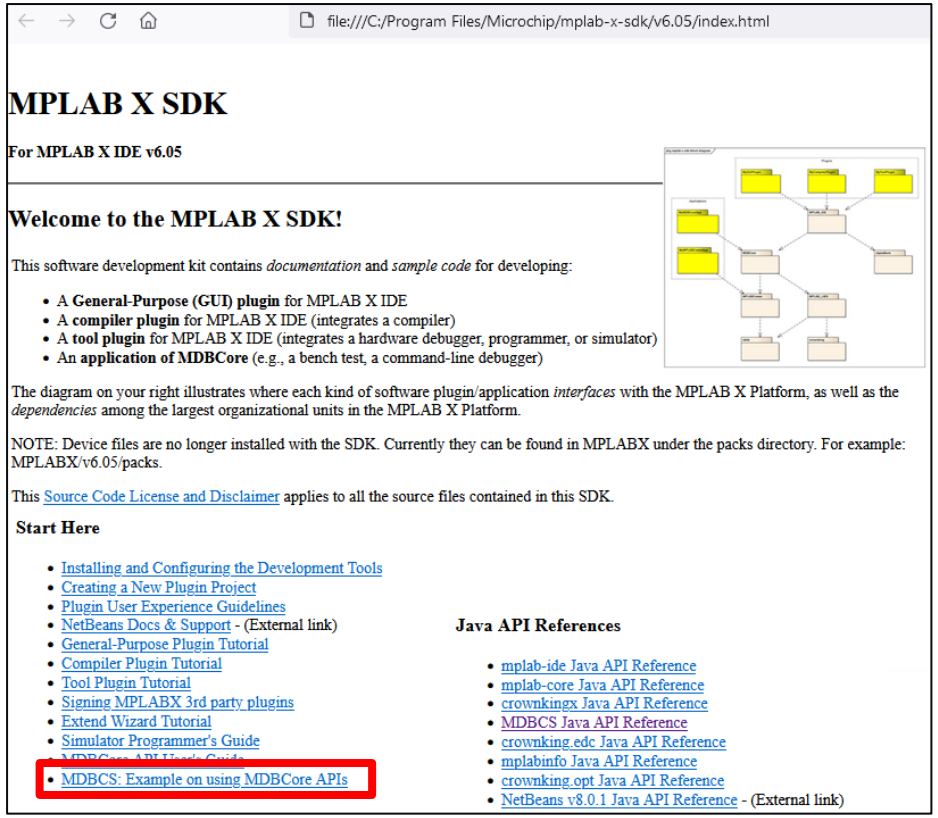
- Netbeans starting ok?
  - Netbeans installation - eg. NB-v8.2
    `C:\Program Files\NetBeans\v18\bin\netbeans64.exe'`

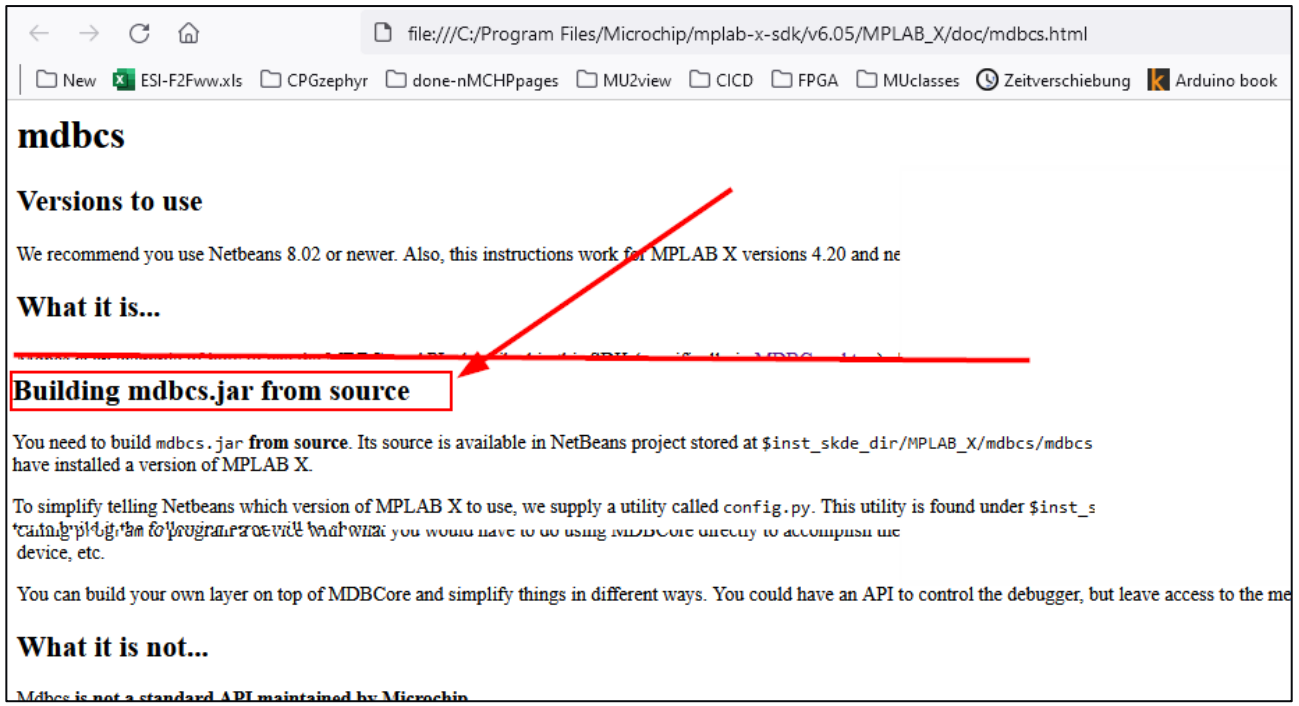


M3b/mdbcs: creating *mdbcs*

Third, how to build **mdbcs** ?

- Create *mdbcs.jar* from Netbeans-project, as described in SDK-docu '**Building mdbcs.jar from source**' (-> <mplab-x-sdk>\index.html)
- Basically load *mdbcs* prj in NB -> set environment -> and compile *mdbcs.jar*
- Obviously with source-code of *mdbcs* , you can improve,change,extend to create your own *mdbcs*





M3b/mdbcs: creating *mdbcs*

...continue 3.step creating *mdbcs*

- loading *mdbcs* project
  <mplab-x-sdk>\MPLAB_X\mdbcs\mdbcs\
  in Netbeans triggers errors on the right

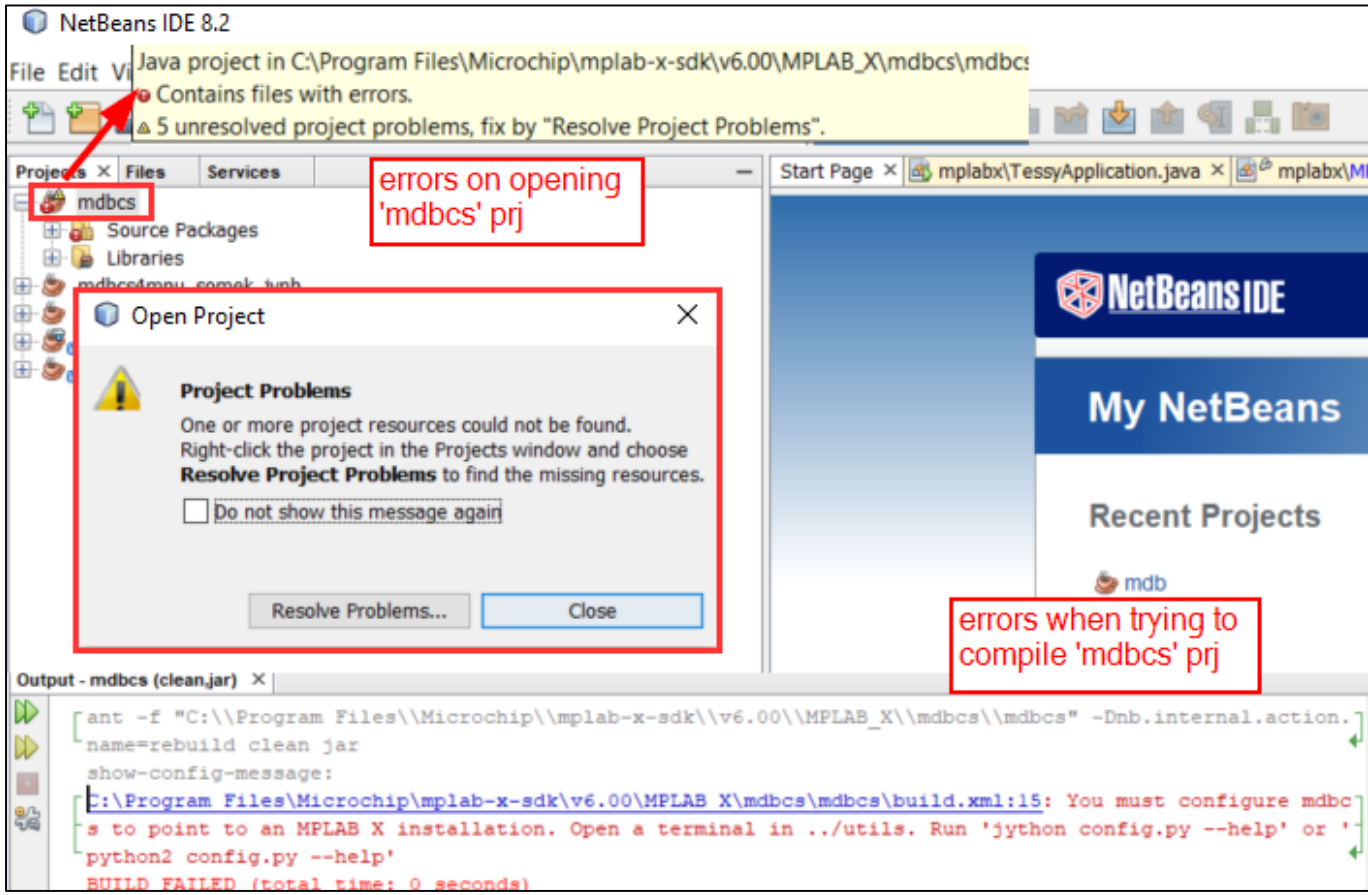
- message in output shows how to fix the error

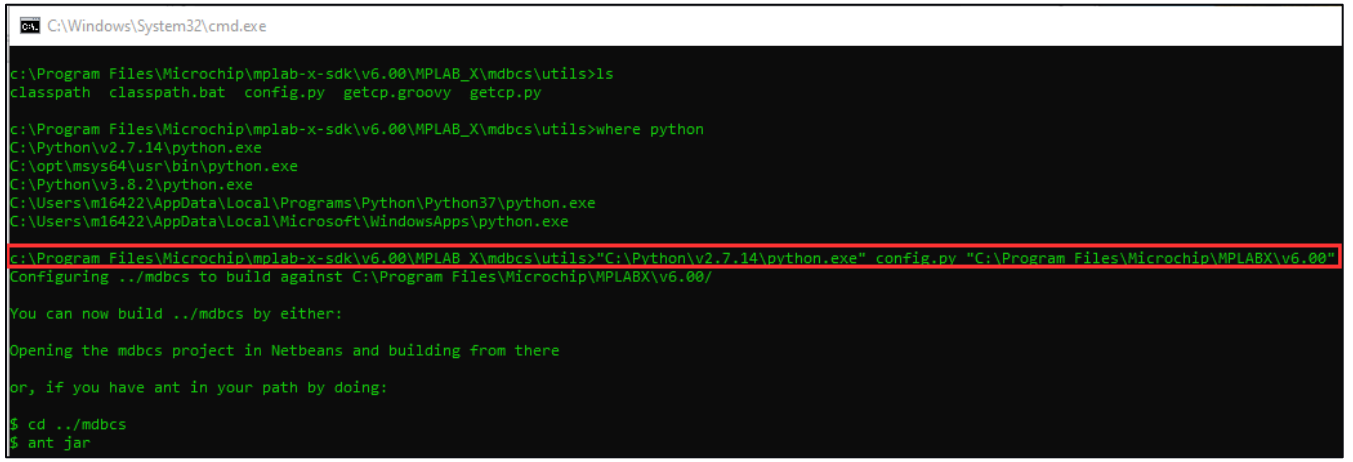
- Basically need to provide path to ***your*** MPLABX installation (might not be default location)

- SDK provides tool *config.py* in
  <mplab-x-sdk\v6.15\MPLAB_X\mdbcs***utils***\>
  to make this step easy with this call

(...\utils\)> python2 config.py <path-to-MPLABX>

- Result is an update of Netbeans-config file
  <mplab-sdk>\MPLAB_X\mdbcs\mdbcs\nbproject\build-impl.xml



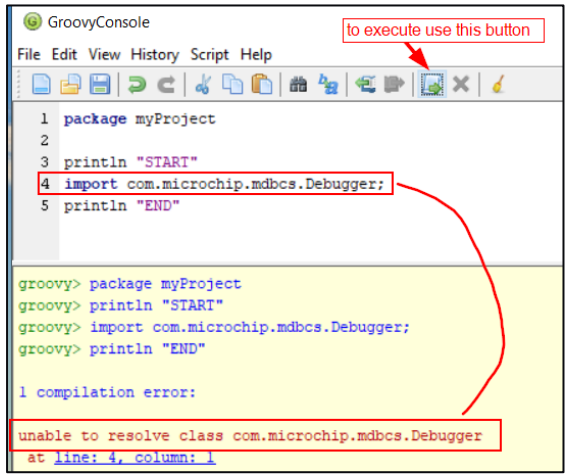


M3b/mdbcs: creating *mdbcs*

...continue 3.step creating *mdbcs*: setting up the CLASSPATH

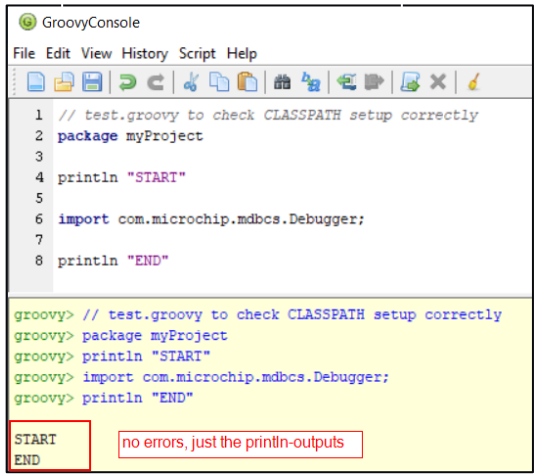
- The `CLASSPATH` = OS-searchpath a list of directories where to find java-module for '`import myJavaModule`' statement in a groovy-script
- The SDK has another helper-script `getcp.py` to help creating the CLASSPATH
- Create your CLASSPATH with
  `(<path-to-mplab-sdk>\MPLAB_X\mdbcs\utils)> python2 getcp.py "<path-to-your-MPLABX-inst>"`
- !! Note the quotes to handle spaces in pathname !! -> output `classpath.bat`
- Test again with
  - in DOS-shell execute *classpath.bat* and start groovyconsole
  - If CLASSPATH ok, you should not get an error anymore

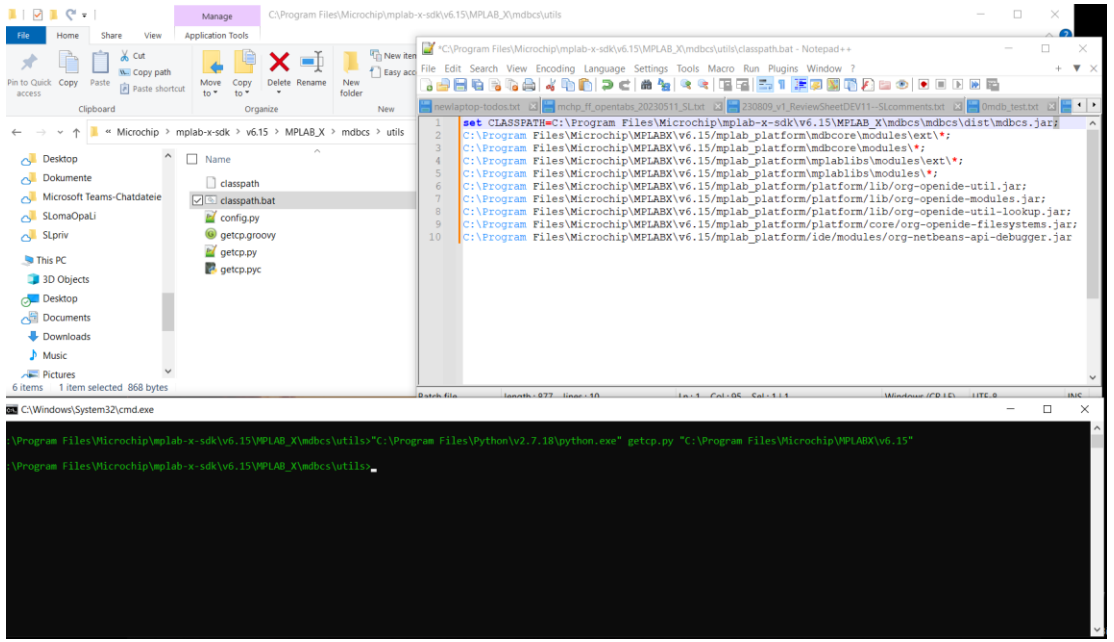
CLASSPATH **not** ok



===>

CLASSPATH **ok**





M3b/mdbcs: summary of preparation to use *mdbcs*

Setup summary

->1: all tools installed?

=> Groovy-v4.0.12, Netbeans-v8.2,
MPLABX-v6.15, MPLABX-SDK-v6.15

->2: all paths defined in OS-searchpath setup?

=>(DOS)> where groovysh ; where java

->2: configuration - CLASSPATH setup?

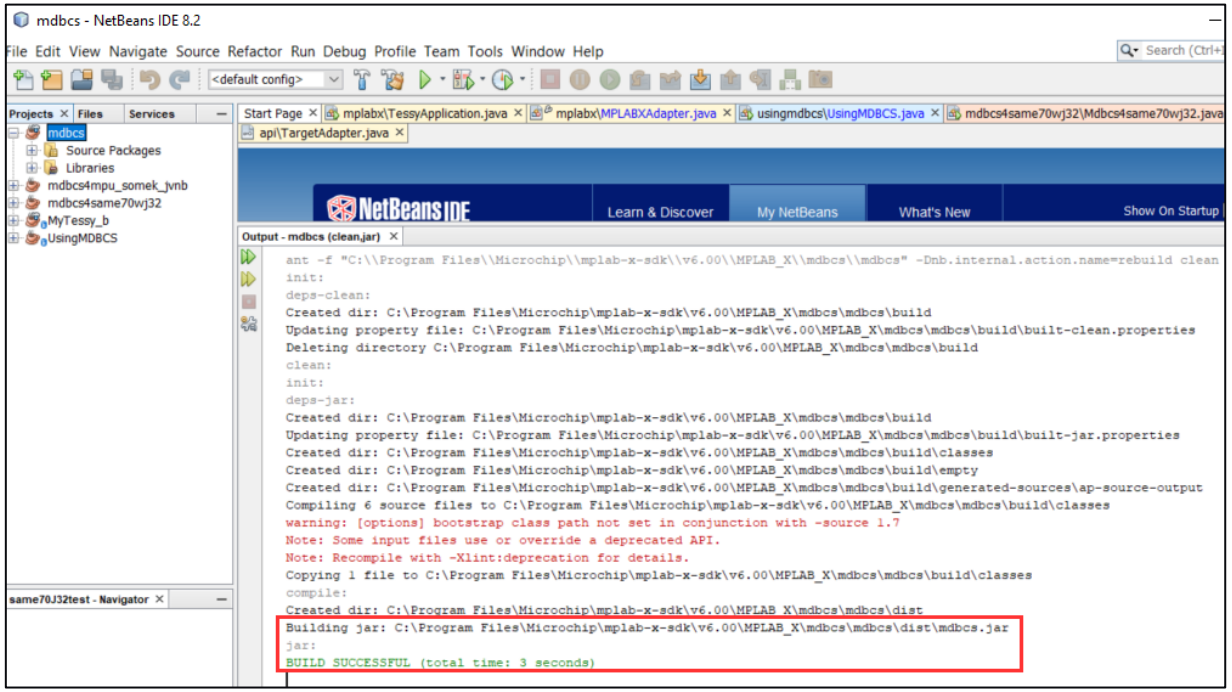
=> 'test.groovy' executes without errors in Groovyconsole

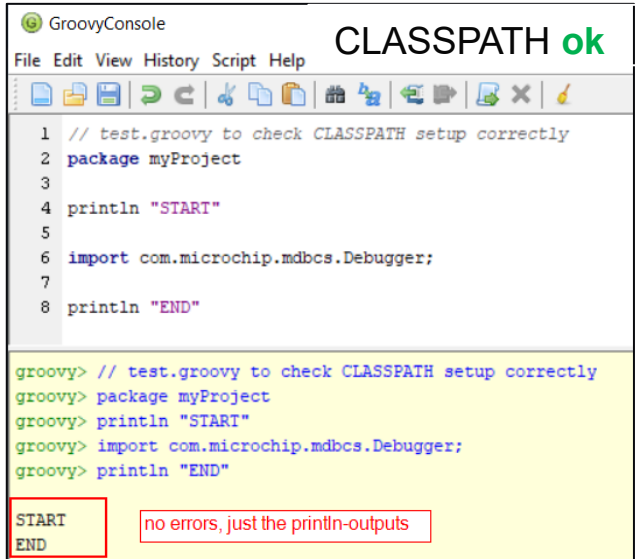
->or with the next step

->3: can you build mdbcs from Netbeans-project ?

=> first compile failes as MPLABX-path not known
-> use <sdk-inst>\MPLAB_X\mdbcs\utils\config.py to set

=> now mdbcs NB-prj should successfully build mdbcs.jar in
<sdk-inst>\MPLAB_X\mdbcs\mdbcs\dist ?





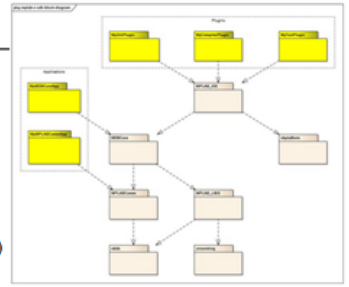
M3b/mdbcs: mdbcs API

mdbcs API from SDK-documenation shows what functions you can use in your script

file:///C:/Program Files/Microchip/mplab-x-sdk/v6.05/index.html

MPLAB X SDK

For MPLAB X IDE v6.05



Welcome to the MPLAB X SDK!

This software development kit contains *documentation* and *sample code* for developing:

- A **General-Purpose (GUI) plugin** for MPLAB X IDE
- A **compiler plugin** for MPLAB X IDE (integrates a compiler)
- A **tool plugin** for MPLAB X IDE (integrates a hardware debugger, programmer, or simulator)
- An **application of MDBCore** (e.g., a bench test, a command-line debugger)

The diagram on your right illustrates where each kind of software plugin/application *interfaces* with the MPLAB X Platform, as well as the *dependencies* among the largest organizational units in the MPLAB X Platform.

NOTE: Device files are no longer installed with the SDK. Currently they can be found in MPLABX under the packs directory. For example: MPLABX/v6.05/packs.

This Source Code License and Disclaimer applies to all the source files contained in this SDK.

Start Here

- Installing and Configuring the Development Tools
- Creating a New Plugin Project
- Plugin User Experience Guidelines
- NetBeans Docs & Support - (External link)
- General-Purpose Plugin Tutorial
- Compiler Plugin Tutorial
- Tool Plugin Tutorial
- Signing MPLABX 3rd party plugins
- Extend Wizard Tutorial
- Simulator Programmer's Guide
- MDBCore API User's Guide
- MDBCS: Example on using MDBCore APIs

Java API References

- mplab-ide Java API Reference
- mplab-core Java API Reference
- crownking.edc Java API Reference
- MDBCS Java API Reference
- mplabinfo Java API Reference
- crownking.opt Java API Reference
- NetBeans v8.0.1 Java API Reference - (External link)

file:///C:/Program Files/Microchip/mplab-x-sdk/v6.05/MPLAB_X/mdbcs/javadoc/index.html

All Classes

Debugger
Debugger.SessionType
Helper
MessageMediatorCMDListener
MException
Pin

PACKAGE CLASS TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

com.microchip.mdbcs

Class Debugger

java.lang.Object
com.microchip.mdbcs.Debugger

Method Summary

All Methods | Static Methods | Instance Methods | Concrete Methods

Modifier and Type	Method and Description
void	`attachSCL(java.lang.String fileName)` This method will append to the list of scl files attached to the simulato
java.lang.Boolean	`blankCheck()` Checks if the device is blank.
void	`clearBP(int breakpointNumber)` Pass the int returned by setBP to clear the breakpoint Works only for SessionType DEBUGGER Target must be connected and halted.
void	`connect()` You must call this method before any methods that require target oper

M3b/mdbcs: mdbcs API

Basically, *mdbcs* provides an abstraction of MDBCore Java-classes -> important one is **Debugger**

- **Create new instance**

```
import com.microchip.mdbcs.Debugger;
debugger = new Debugger(…);
```

- **Control**

```
debugger.run()
debugger.halt()
debugger.step()
```

- **Query**

```
debugger.getPC()
debugger.getSymbolAddress(“foo”)
debugger.readFileRegisters(address, length, buffer)
```

- **Access Variables and memory**

```
debugger.writeRegister(“LATD”, 0x0007)
```

- **Check mdbcs source or docu for more**

```
<mplab-x-sdk>/MPLAB_X/mdbcs/javadoc/index.html
```

- You miss a functionality? -> SDK provides NB-prj with Java-sources, so create your own !!

M3b/mdbcs: why Groovy?

A word on scripting languages with Java-interface

- Need a scripting-language with a Java-Interface, so we can access MDBCore-functionality (=Java)
- See SDK-docu '**When to use this mdbcs example**' (-> mdbcs.html) for reasons when to use *`mdbcs`* over *`mdb`*
- SDK has examples with
  - Scripting: ***Jruby*** , ***Jython***
  - Programming: ***Java*** (more complex Java-code then simple scripting ->see module3c/Java,MDBCore)
- ***Groovy*** selected for this class -> what is it and why Groovy?
  - Current SDK-v6.15 has no SDK-examples in Groovy yet -> will come with future SDK-releases
  - Dynamically typed language (= no variable-type declarations -> same for Jruby, Jython)
  - Has native Java-interface, so can directly call into MDBCore-Java modules and back
  - ...

M3b/mdbcs: Groovy in a nutshell

Comments

Name of your Java package

Import of used classes

(from CLASSPATH)

Create class/objects

with member variables
(typeless variable *name*)

and function with for-loop

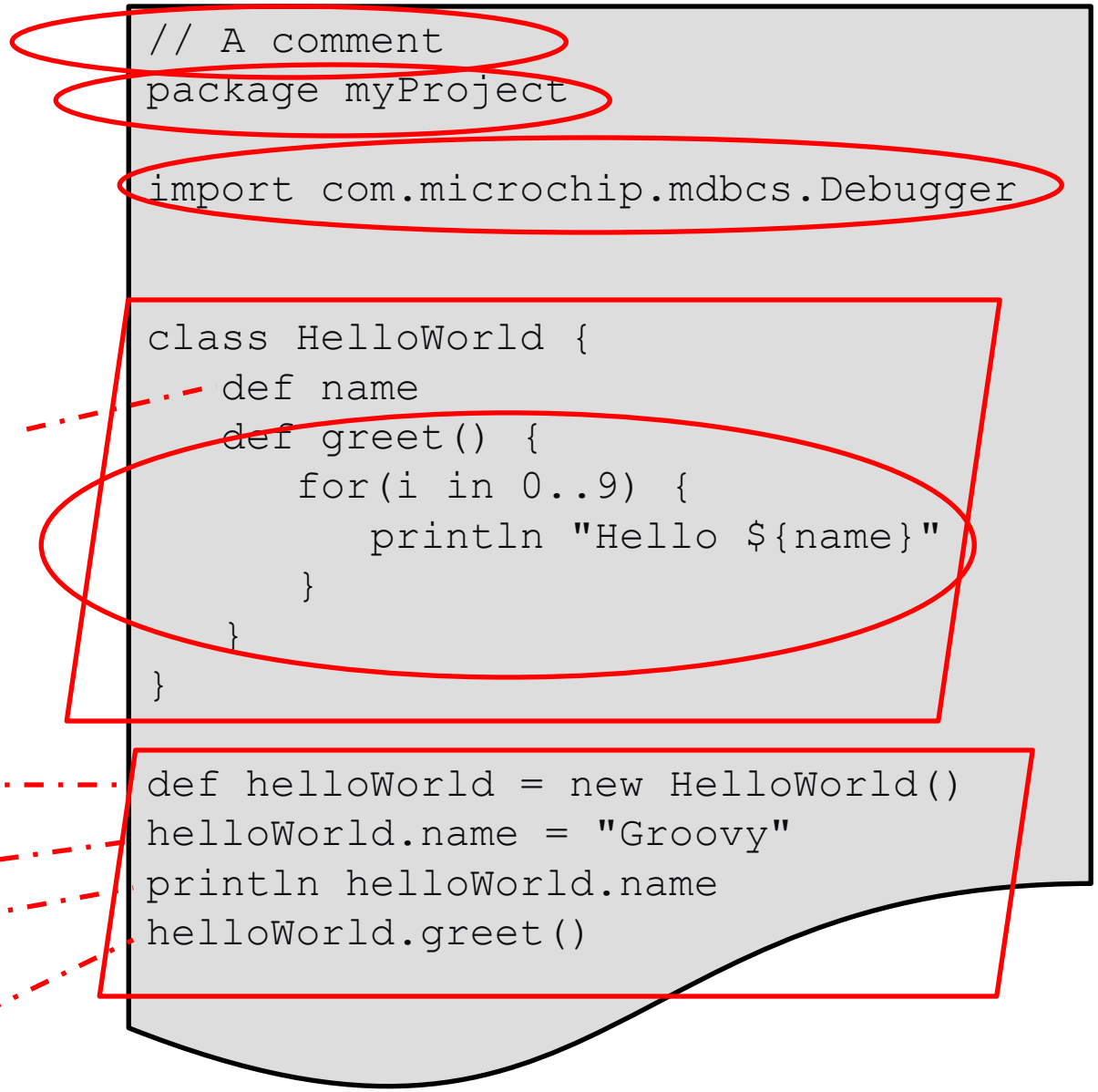
Scripting language

creation of objects

Initializing members

simple IO

call member functions



```
// A comment
package myProject

import com.microchip.mdbcs.Debugger


class HelloWorld {
    def name
    def greet() {
        for(i in 0..9) {
            println "Hello ${name}"
        }
    }
}

def helloWorld = new HelloWorld()
helloWorld.name = "Groovy"
println helloWorld.name
helloWorld.greet()
```

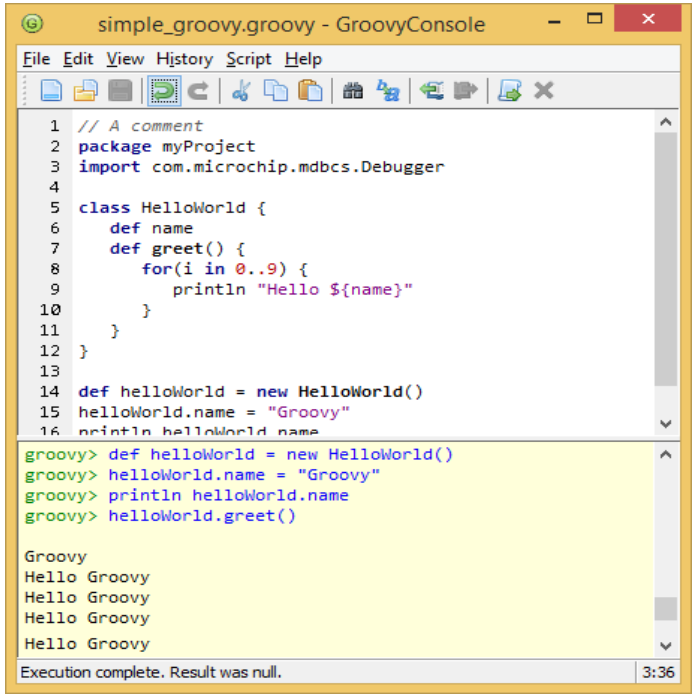
M3b/mdbcs: Groovy usage

choices when testing&developing Groovy scripts

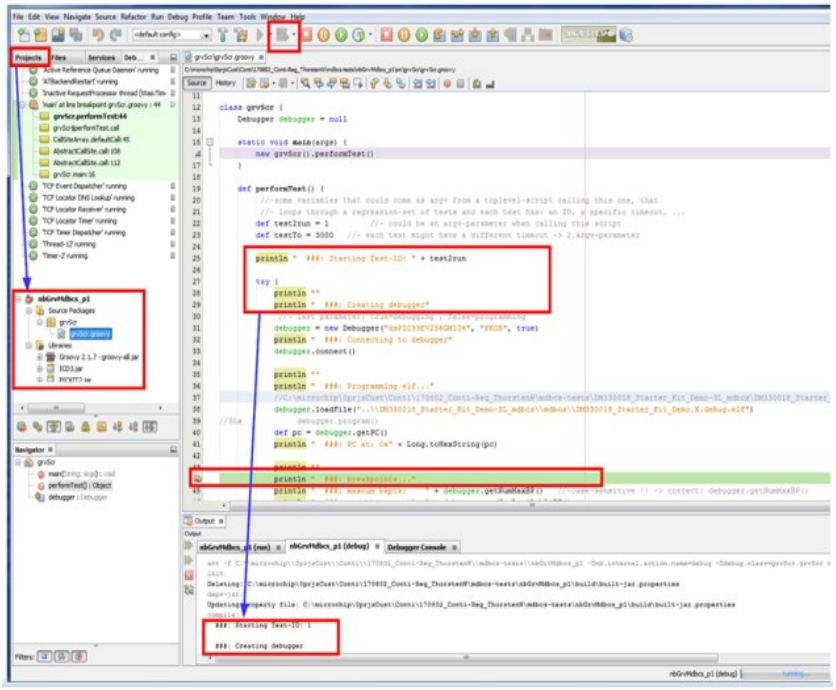
(DOS)> groovysh

```
groovy:000> class HelloWorld {
groovy:001> def greet() {
groovy:002> println "Hello"
groovy:003> }
groovy:004> }
===> true
groovy:000> x = new HelloWorld()
===> HelloWorld@606e76b4
groovy:000> x.greet()
Hello
===> null
groovy:000>
groovy:000> :q
```

(DOS)> groovyconsole



(DOS)> netbeans



M3b/mdbcs: scripting styles

2 styles: Procedural-style and ObjectOriented-style

- Procedural-style: simple, less coding, no try-catch
  -> must complete entire script
- ObjectOriented-style:more coding,
  try-catch -> error-handling and object-cleanup
- API-description in SDK
- Comparison of both styles on next slide

'procedural' style

```
//-myTest2.gr
package myProject
import com.microchip.mdbcs.Debugger;

debugger=new Debugger("dsPIC33EV256GM106",
                      "pkob", true)
ELF2USE=./debug/SAMC21J18A.X.debug.elf
debugger.connect()
debugger.loadFile(ELF2USE)
debugger.program()
...
```

To exec:

```
(DOS)> classpath.bat
(DOS)> groovy myTest2_script.gr
```

set classpath
->previous slide

'object-oriented' style

```
 //-myTest1.gr
package myTest1Pck
import com.microchip.mdbcs.Debugger
class myTest1 {
    Debugger debugger = null
    static void main(args) {
        new myTest1().performTest()
    }
     //- perform the test
    def performTest() {
      try{
         println "  ###: Creating debugger"
         debugger = new debugger("dsPIC33…", "PKOB", true)
         debugger.connect()
         ...
      } //-end 'try'
      catch(e){
         ...
         debugger.disconnect()
         debugger = null
      } //-end 'catch'
    } //-end 'def performTest()'
} //-end 'class myTest1'
```

To exec:

```
(DOS)> classpath.bat
(DOS)> groovy myTest2_proc.gr
```

M3b/mdbcs: scripting styles

Compare Groovy-script styles:

Procedural style

```
package myTest1Pck
import com.microchip.mdbcs.Debugger

println "  ###: Creating debugger"
debugger = new Debugger("SAML21","EDBG", true)
res=debugger.connect() //='0' if successful
if(!res)
   debugger.disconnect()
   debugger = null
else
   debugger.loadFile("myprog.elf")
   debugger.program()
```

To exec with *groovysh*

```
(DOS)> classpath.bat
(DOS)> groovy myTest2.gr
```

To exec with *groovyconsole*

```
(DOS)> classpath.bat
(DOS)> groovyconsole
(groovyconsole)> open+exec myTest2.gr
```

Or as project within Netbeans

ObjectOriented style

```
package myTest1Pck
import com.microchip.mdbcs.Debugger
class myTest1 {
    debugger Debugger = null
    static void main(args) {
        new myTest1().performTest()
    }
     //- perform the test
    def performTest() {
      try{
         println "  ###: Creating debugger"
         debugger = new Debugger("SAML21","EDBG",true)
         debugger.connect()
         debugger.loadFile("myprog.elf")
         debugger.program()
      } //-end 'try'
      catch(e){
         ...
         debugger.disconnect()
         debugger = null
      } //-end 'catch'
    } //-end 'def performTest()'
} //-end 'class myTest1'
```

M3b/mdbcs: mdb vs mdbcs

Comparing mdbbat-cmdfile with mdbcs-version in Groovy-script

mdbbat command file

```
//(DOS)> mdb.bat thisfile.txt
device ATSAME70Q21B


set communication.interface swd

hwtool EDBG


Reset
program "fwDev11_same70_xult.X.debug.elf"


break endOfTest
run
wait
halt

print testResult

quit
```

Groovy script using mdbcs (procedural-style)

```
//(DOS)> groovysh thisfile.groovy
package myTest1Pck
import com.microchip.mdbcs.Debugger
DEV2USE=ATSAME70Q21B
DBG2USE=EDBG
Properties props =  debugger.getToolProperties();
props.setProperty("communication.interface", "swd");

debugger = new Debugger(DEV2USE, DBG2USE , true)
debugger.connect()

debugger.reset
debugger.loadFile("fwDev11_same70_xult.X.debug.elf")
debugger.program()

debugger.setBP("endOfTest")
debugger.run
debugger.halt()
debugger.isRunning()){}

def addrAv = debugger.getSymbolAddress("testResult")
byte[] aveVal = new byte[4] //-32bit MCU: word=4B
debugger.readFileRegisters(addrAv, aveVal.length, aveVal)
println "Value 0x" + aveVal

debugger.disconnect()
debugger = null
```

M3b/mdbcs: mdb vs mdbcs

Comparing mdbbat-cmdfile with mdbcs-version in Groovy-script

mdbbat command file

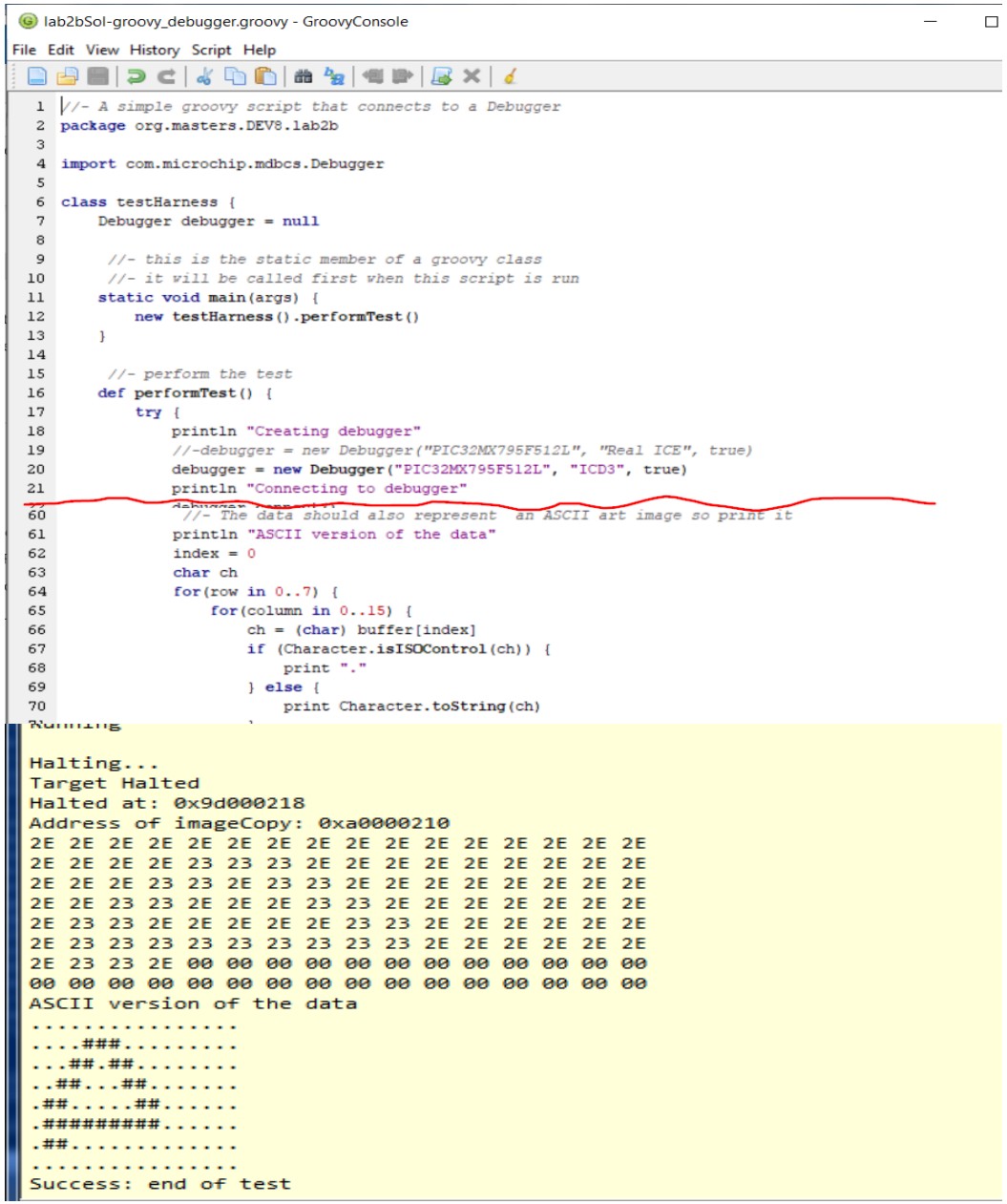
```
//(DOS)> mdb.bat thisfile.txt
device ATSAME70Q21B


set communication.interface swd

hwtool EDBG


Reset
program "fwDev11_same70_xult.X.debug.elf"


break endOfTest
run
wait
halt

print testResult

quit
```

Groovy script using mdbcs (procedural-style)

```
//(DOS)> groovysh thisfile.groovy
package myTest1Pck
import com.microchip.mdbcs.Debugger
DEV2USE=ATSAME70Q21B
DBG2USE=EDBG
Properties props =  debugger.getToolProperties();
props.setProperty("communication.interface", "swd");

debugger = new Debugger(DEV2USE, DBG2USE , true)
debugger.connect()

debugger.reset
debugger.loadFile("fwDev11_same70_xult.X.debug.elf")
debugger.program()

debugger.setBP("endOfTest")
debugger.run
debugger.halt()
debugger.isRunning()){}

def addrAv = debugger.getSymbolAddress("testResult")
byte[] aveVal = new byte[4] //-32bit MCU: word=4B
debugger.readFileRegisters(addrAv, aveVal.length, aveVal)
println "Value 0x" + aveVal

debugger.disconnect()
debugger = null
```

M3b/mdbcs: how mdbcs solves mdb limits

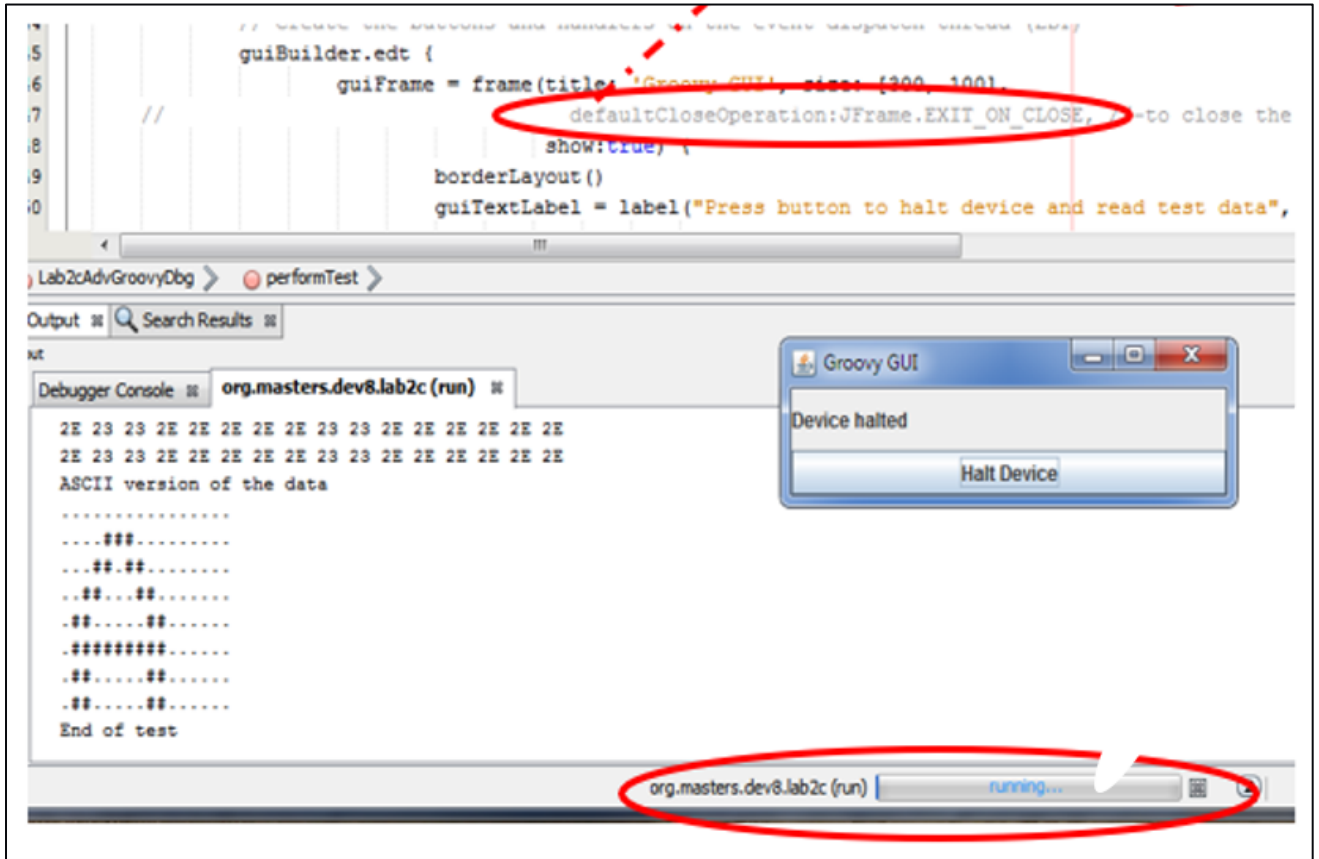
- Why `mdbcs`? How to overcome `mdb` limits ?
- `mdb_cmd.txt` does not support features like
  - if *.elf doesn't exist:
    `if (!elf_file_exists) then compile-from-src`
  - no looping `for`, `while`
  - output (`mdb` 'print') only printed to stdout,
    so cannot do something like this in testing:
    `if (mdb_print_of_result>10) then seed=30`
    `else seed=15`
    ->the demo for this module uses this capability
  - another example shown right. Assume you're printing some memory which is actually an ASCII-art picture (lower print) -> with `mdb` you can only print the hex-values read, whereas with `mdbcs` you can print the ASCII-chars instead, hence see ASCII-art picture immediately instead postprocess
  - As a scripting-language is needed so `mdbcs.jar` can be imported, any other Java-library can be used, like SwingUI shown next



M3b/mdbcs: potentials with mdbcs

- `mdbcs.jar` is just a Java-module
- It must be imported into the used scripting language with `import mdbcs.*`
- Therefore the used scripting language needs to have a Java-interface, like Jython, Groovy
- But with that Java-IF you can also import any other Java-library
- One interesting Java-lib is the Swing-UI which allows to create simple UIs
- You can think of creating a small UI, that shows the progress of your current regression

Image the choices you have !!



M3b/mdbcs: FW used for Lab

```c
int main ( void )
{
    /* Initialize all modules */
    SYS_Initialize ( NULL );
    printf("\n\r");
    printf("\n\r START CICDtesting ...");

     //-SL: inits
    myCnt=0; //-SL: central counter
    finalLedCnt=0;
    sysTickTmrExFlag=false; //-SL: SysTickWrap-event flag
     //-SL: SW0 defined as GPIO-In but still need to call this fct to enable In
    SW0_InputEnable();
    char2prntIdx=0; //- default char-to-print = 'b'

    SYSTICK_TimerStart();
    SYSTICK_TimerCallbackSet(&sysTickTimeout_handler, (uintptr_t) NULL);

    while ( true )
    {
        /* Maintain state machines of all polled MPLAB Harmony modules. */
        if(sysTickTmrExFlag)
        {
            sysTickTmrExFlag=false;
            LED0_Toggle();
            finalLedCnt++;

            myCnt++;
            if(cEchoCnt/2>=myCnt)
            {
                printf("a");
            } else if ((cEchoCnt/2<myCnt) & (cEchoCnt>=myCnt))
            {
                //printf("b");
                printf("%c",charArray[char2prntIdx]);
                if (cEchoCnt==myCnt)
                    myCnt=0;
            } else
                printf("ERROR1");
        }

        if(SW0_Get() == PRESSED)
        {
            testEnd();
        }
    }

    /* Execution should not come here during normal operation */

    return ( EXIT_FAILURE );
}
```

```c
volatile uint32_t myCnt;
volatile uint32_t finalLedCnt;
#define cEchoCnt 10
#define PRESSED  0
#define RELEASED 1
volatile bool sysTickTmrExFlag;

char charArray[3] = {'b','c','d'};
volatile uint32_t char2prntIdx; //- index for 'charArray'
void testEnd(void);


void sysTickTimeout_handler(uintptr_t context)
{
    sysTickTmrExFlag=true;
}
```

```
COM8:115200bps - Tera Term VT
File  Edit  Setup  Control  Window  Help

START CICDtesting ...aaaaabbbbbaaaaabbcccaaaaaccccccaaaa
aaaccccccaaaaaccccccaaaaaccccccaaaaac
testEnd with finalLedCnt=116
```

```c
void testEnd(void)
{
    printf("\n\r testEnd with finalLedCnt=%d\n\r ",(int)finalLedCnt);
    printf("\n\r");
    while(1){}
}
```

Three FW functionalities:

1. LED blinks @SYSTICK=250ms

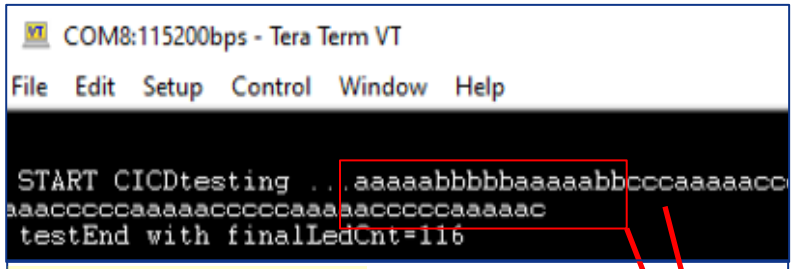
2. echo a char on uart-console
   - 1.char printed is always 'a'
   - 2.char selected from array `charArray` with variable `char2prntIdx`
   - print 5x1.char/'a' and 5x'2.char/'b,c,d' (default 'b')

   => changing `char2prntIdx` shows that you can change FW behaviour from the script

3. on HW-button SW0 press
   - it jumps to `endOfTest()`
   - in `endOfTest()` print the value of total LEDtoggles stored in `finalLedCnt`
   - and loop there

M3b/mdbcs: advantages of mdbcs vs mdb

To understand where *mdbcs* can help, quick recall from mod3a/*mdb*, what the demo does




```
device same54
hwtool EDBG
program path\myfw.elf
break endOfTest()
run
wait 4500
 #- charIdxCnt={0=b, 1=c, 2=d}  -> init=0 in FW so
 #-    prints aaaaabbbbbaaaaabb…
 #-  -> now stop ***manually***
halt
print finalLedCnt
  18
print charIdxCnt
 0 #- == FW-init -> manually change to '1'='c'
write /p charIdxCnt 1

continue
wait 5500
 #-now prints 'aaaaacccccaaaaacc…'
halt
 #-stopped after 5.5s OR if SW0=pressed
quit
```

FW-functionality used in demo and expect console output running mdb_cmd.txt on the left

1. FW-functionality:
   a. toggleLED@500ms -> finalLedCnt++@250ms (both edges On+Off)
   b. 1xchar on UARTout@250ms with default charIdxCnt=0 init'd in FW
   c. 3.fct is to stop on SW0=press

What does the *mdb_cmd.txt* on the left do?

1. set device -> connect EDBG -> download *.elf
2. set breakpoint to symbol/function 'endOfTest()'
3. start ***run*** and wait(4,5s)
4. toggleLED@500ms -> finalLedCnt++@250ms (both edges On+Off)
   -> so after 4.5s/250ms, the value of finalLedCnt= after 4.5s/250ms=18
5. uartOut@250ms starts with default charIdxCnt=0 hence 2.char='b' and 4.5s = 18x chars, so the console shows: 'aaaaa.bbbbb.aaaaa.bb(b)' (!! last 'b' not echo'd) and halt
6. change charIdxCnt=1 which means 2.char printed now is 'c', hence after the first 17x char from first 4.5s, the next char (!!18.char from first 4.5s not printed!) is 'c'
7. Then continue with 'c' as 2.char so you get 'cccaaaaacccccaa…' for 5.5s or SW0-press

Next we'll see how this can automated with mdbcs

M3b/mdbcs: advantages of mdbcs vs mdb

Groovy script using `mdbcs` and interpreting the output => solving mdb.bat limit

```
package myCICDmdbcs
import com.microchip.mdbcs.Debugger;

class myTest {
    def Debugger debugger = null;
    static void main(args) {
        new myTest().run();
    } //-end 'void main()'
     //-perform test
    def run() {
        try {
            println("------------- MAIN -----------------");
            println("##### Start-2 running-" + ELF2USE + "-");
            Properties props = debugger.getToolProperties();
            props.setProperty("programoptions.eraseb4program", "true");
            debugger = new Debugger("ATSAMD21J18A", "EDBG", true);
            debugger.connect();
            debugger.loadFile(ELF2USE);
            debugger.program();
```

start as discussed

```
            //-##### 1.run for random-num of millis  #####-
            Random randVal = new Random()
            int val3 = (randVal.nextInt(10)+2) //-at least run 2s
            int randMillis = val3*1000

            debugger.run();  //-run and halt after 'random-milli-sec'
            debugger.sleep(randMillis);
            debugger.halt();
            while (debugger.isRunning()) {};
            def pc = debugger.getPC();
            println("############ Halted at: 0x" + Long.toHexString(pc));
```

run1 uses random-time

```
//-##### 2.run for random-num of millis  #####-
//- goal for 2.run to solve mdb.bat-limit by 'interpreting output'
//-  ==> in 1.run UARTTX prints 2 chars, ch1='a' fix and
//-        ch2=array[char2prntIdx]={'b','c','d'} initialized to '0'
//-  ==> Now for 2.run following condition to decide if ch2='c' or ='d'
//-     ch2='c' if '0                < $(finalLedCnt)%10 < $(REGR_LED_THR)' (== 0< finalLedCnt%10 <5)
//-     ch2='d' if '$(REGR_LED_THR) < $(finalLedCnt)%10 < $(REGR_LED_MAX)' (== 6< finalLedCnt%10 <9)
//- read current value of global variable 'finalLedCnt'
String finalLedCntSymbStr = "finalLedCnt";
def finalLedCntAddr = debugger.getSymbolAddress(finalLedCntSymbStr)
byte[] finalLedCntBuf = new byte[4]
debugger.readFileRegisters(finalLedCntAddr, finalLedCntBuf.length, finalLedCntBuf);
def finalLedCntVal = byte2long(finalLedCntBuf);
println("############ finalLedCnt:" + finalLedCntVal);
```

read variable 'finalLedCnt'

```
def finalLedCntValTest = finalLedCntVal%LED_CNT_MOD   //-calc modulo
def CH2_IDX_NEW = 0
if ( (0 <= finalLedCntValTest) && (finalLedCntValTest <      5      ) )
{
    CH2_IDX_NEW = 1
} else if ( (      5      <= finalLedCntValTest) && (finalLedCntValTest <      10      ) )
{
    CH2_IDX_NEW = 2
} else
{
    CH2_IDX_NEW = 0
}
```

use modulo

evaluate 'finalLedCnt' to decide what 2.char should be

```
//-#### finally write decision == new 'char2prntIdx' for 2.run into HW
byte[] ch2IdxNewBuf = new byte[4]
ch2IdxNewBuf = long2byte(CH2_IDX_NEW);
debugger.writeFileRegisters(char2prntIdxSymbAddr, ch2IdxNewBuf.length, ch2IdxNewBuf);
```

now set char2 = write charArray-Index

```
//-continue running with new 'char2prntIdx'
int val4 = (1 + randVal.nextInt(10))
int rand2Millis = val4*1000
debugger.run();
debugger.sleep(rand2Millis);
debugger.halt();
while (debugger.isRunning()) {};
```

calc another random time and continue

OR check for SW0-press

```
//-if user pressed SW0 during run the FW would have jumped to 'endTest()'
String testEndSymb = "testEnd"
debugger.setBP(testEndSymb)
def testEndAddr = debugger.getSymbolAddress(testEndSymb)
debugger.setPC(testEndAddr)

println("##### ======= Success: end of test =======##");
// Tidy up
debugger.disconnect()
debugger = null
System.exit(0)
} //-end 'try'
catch(e) {
    println "=====================================================";
    println "================ SL: exception ====================";
    println "== Ex.class= " + e.getClass();
    println "== Ex.cause= " + e.getCause();
    println "== Ex.msg= " + e.getMessage();
    println "== Ex.stackTrace= " + e.getStackTrace();
}
```

cleanup

catch any error along the way

M3b/mdbcs: demo for mdbcs

mdbcs demo

M3b/mdbcs: Lab for mdbcs -> how to get&run lab

Steps to get+run the example for this mod4/mdbcs

1. assumptions (same as for mod3a/mdb)
   - you have installed MPLABX-v6.15 and XC32-v4.35 (latest today Dec'23) in standard-paths
   - your OS-searchpath contains "<mplabx>\mplab_platform\bin:<mplabx>\gnuBins\GnuWin32\bin"
   - you have some version of git installed on your system OR you manually pull the repository
   - you have the required HW atsamd21-xpro and it is attached via USB to your host
   - you have any terminal program installed, like TerraTerm
   - in addition, for mdbcs you need (as described on slides on 'preparation to use *mdbcs*')
     - create mdbcs.jar: get MPLABX-SDK -> get+install Netbeans-v8.2 -> load mdbcs into NB -> provide MPLABX-path (-> *<sdk-inst>\MPLAB_X\mdbcs\utils\config.py*) -> compile *mdbcs.jar*
     - get+install Groovy and set JAVA_HOME -> test: `(DOS)> where groovysh ; where java`

2. first clone git repository local with

   #-> open a DOS-console, cd into any directory where you want to run and use git to pull the repository

   ```
   (DOS:)> mkdir myDir ; cd myDir
   (DOS:myDir\)> git clone https://github.com/stefanluethin-microchip/mu_dev11_mdbcore
   ```

   #->will create new directory 'mu_dev11_mdbcore\' OR create directory and get repo manually

M3b/mdbcs: Lab for mdbcs -> how to get&run lab

Steps to get+run the example for this mod4/mdbcs (continued)

3. The repo contains a txt-file with the steps to run
   `mu_dev11_mdbcore\dev11Labs_mdbScripts\dev11mod4Lab_mdbcs-0-0steps_2_run.txt`

4. next cd into new directory `mu_dev11_mdbcore\` and checkout git-tag *final_4_dev11-mod4_mdbcs*

5. if did not prior run mod3a/mdb lab, you must compile your FW first with two steps, otherwise skip
   #->update/create Makefiles with MDBCore-module `prjMakefilesGenerator.jar` in bat-script:
   ```
   (DOS:mu_dev11_mdbcore\)> prjMakefilesGenerator.bat .\dev11Labs_fw\firmware\CICDgit_samd21xplp.X
   ```
   #->compile using `make`
   ```
   (DOS:mu_dev11_mdbcore\)> cd .\dev11Labs_fw\firmware\CICDgit_samd21xplp.X && make && cd ..
   ```

5. prepare to use `mdbcs`
   #->setup OS-search -> first need to save current PATH into `BASEPATH` -> script stops if `BASEPATH` not set:
   ```
   (DOS:mu_dev11_mdbcore\)> set BASEPATH=%PATH%     #-save env-path %PATH% and then extend
   (DOS:mu_dev11_mdbcore\)> .\dev11Labs_mdbScripts\dev11mod4Lab_mdbcs-1Setup_env.bat
   ```
   #->setup CLASSPATH using script created in preparation, just renamed
   ```
   (DOS:mu_dev11_mdbcore\)> .\dev11Labs_mdbScripts\dev11mod4Lab_mdbcs-2Classpath.bat
   ```

6. finally start groovy-call `groovy dev11mod4Lab_mdbcs-scr.groovy` one

M3b/mdbcs: Lab for mdbcs -> how to get&run lab

Steps to get+run the example for this mod4/mdbcs (continued): Summary

All these steps 1-6 are wrapped into one bat-script (setup_env -> classpath -> groovy-call):

#-save env-path %PATH% and then extend

```
(DOS:mu_dev11_mdbcore\)> set BASEPATH=%PATH%
```

#-start terminal to see output

```
(DOS:mu_dev11_mdbcore\)> "C:\Program Files (x86)\teraterm\ttermpro.exe"
```

#-start mdbcs-script

```
(DOS:mu_dev11_mdbcore\)> .\dev11Labs_mdbScripts\dev11mod4Lab_mdbcs-run.bat
```

Before we run the demo a few more things to pay attention to on the next slides,
so it becomes clear why *mdbcs* outstands *mdb*

M3b/mdbcs: why *mdbcs* outstands *mdb*

Steps to get+run the example for this mod4/`mdbcs` (continued)

Running the demo: this mdbcs-demo essentially does the same as the mdb-demo in mod3b/mdb ***BUT***

- with `mdb.bat` user had to
  - run a fixed time and ***manually*** `halt` the `mdb.bat` run
  - then ***manually*** change the variable `char2prntIdx`
  - and ***manually*** restart, to see the effect of the change
- with `mdbcs` you can automate this (->check Groovy-script)
  - it uses random()- and modulo()-functions to run for a random time instead of fixed time and halts
  - once halted get random-time we ran (==variable `finalLedCnt`) automatically AND based its value set variable `char2prntIdx` to
    - =1 -> ch2='c' if ' 0 < $(finalLedCnt)%10 < $(REGR_LED_THR)' (->finalLedCnt=0-4)
    - =2 -> ch2='d' if '$(REGR_LED_THR) < $(finalLedCnt)%10 < $(REGR_LED_MAX)' (->finalLedCnt=5-9)

->this ability to react on results is one advantage of `mdbcs` besides the possibilities to use any Java-module and the features of a scripting language…

M3b/mdbcs: Lab for mdbcs -> expected result

Steps to get+run the example for this mod4/mdbcs (continued)

10. ... analysis of groovy-script to identify mentioned parts (part1: startup, env)

```
c:\Test\dev11_mod4_mdbcs_test\mu_dev11_mdbcore>.\dev11Labs_mdbScripts\dev11mod4Lab_mdbcs-3run.bat
_
######: ================================================================
######(dev11mod4Lab_mdbcs-run.bat): Starting from c:\Test\dev11_mod4_mdbcs_test\mu_dev11_mdbcore\dev11Labs_fw\firmware\CICDgit_samd21xplp.X
######                             groovy ..\..\..\dev11Labs_mdbScripts\\dev11mod4Lab_mdbcs-scr.groovy
_
_
######: ================================================================
######: setting up OS-path...
dev11mod4Lab_mdbcs-1Setup_env.bat Starting ...
   ##: BASEPATH set -> continue
   ##: extending OS-searchpath
dev11mod4Lab_mdbcs-1Setup_env.bat: END (errorlevel: 0)
   ..\..\..\dev11Labs_mdbScripts\dev11mod4Lab_mdbcs-1Setup_env.bat run ok
_
######: ================================================================
######: setting up Java-path...
   ..\..\..\dev11Labs_mdbScripts\dev11mod4Lab_mdbcs-2Classpath.bat run ok
_
######: ================================================================
######: checking if elf-2-use is available...
############- from c:\Test\dev11_mod4_mdbcs_test\mu_dev11_mdbcore\dev11Labs_fw\firmware\CICDgit_samd21xplp.X elf2use dist\samd21xplp\production\CICDgit_samd21xplp
X.production.elf does exist -> continue
_
######: Starting mdbcs.groovy from c:\Test\dev11_mod4_mdbcs_test\mu_dev11_mdbcore\dev11Labs_fw\firmware\CICDgit_samd21xplp.X:
######:                           groovy "..\..\..\dev11Labs_mdbScripts\\dev11mod4Lab_mdbcs-scr.groovy"
=============================
##### Start-2 running-dist\samd21xplp\production\\CICDgit_samd21xplp.X.production.elf-
PACKS location (using packs from MPLAB X installation mdbcs was built against) = C:/Program Files/Microchip/MPLABX/v6.00/packs
Thirdparty lib location (using packs from MPLAB X installation mdbcs was built against) = C:/Program Files/Microchip/MPLABX/v6.00/mplab_platform/thirdparty/
May 05, 2023 4:24:33 PM org.openide.util.NbPreferences getPreferencesProvider
WARNING: NetBeans implementation of Preferences not found
[WebSocketSelector-41] ERROR org.java_websocket.server.WebSocketServer - Shutdown due to fatal error
java.net.BindException: Address already in use: bind
        at sun.nio.ch.Net.bind0(Native Method)
```

starting bat-script that call groovy-script

OS-search-path check, set by prior call of

dev11mod4Lab_mdbcs-1Setup_env.bat

CLASSPATH check, set by prior call of

dev11mod4Lab_mdbcs-2Classpath.bat

check if *.elf exists?

groovy call

M3b/mdbcs: Lab for mdbcs -> expected result

Steps to get+run the example for this mod4/`mdbcs` (continued)

10. ... In the 2.part of the groovy-script we do 2 runs with random times:

```
by default. If you wish to read custom ranges, please go to the Memories to Program property page and specify the ranges you wa
Programming complete


================================================================================
              STOP anytime by pressing SW0...
     ############ Performing test (1.run) -> running 6sec and then halt


Running

Halting...
Target Halted
############ Halted at: 0x1670
############ Address of "finalLedCnt": 0x20000008 Value : 22
############ Address of "char2prntIdx": 0x20000004 Value : 0
############ for 2.run ch2_idx(new): 1


================================================================================
              STOP anytime by pressing SW0...
     ############ Performing test (2.run) -> running 5sec and then halt


Running

Halting...
Target Halted
############ Address of "finalLedCnt": 0x20000008 Value : 43

##### ======================================
##### ======= Success: end of test =======##

======(dev11mod4Lab_mdbcs-run.bat): SUCCESS
#######(dev11mod4Lab_mdbcs-run.bat): DONE

c:\Test\dev11_mod4_mdbcs_test\mu_dev11_mdbcore>
```

```
COM37:115200bps - Tera Term VT
File  Edit  Setup  Control  Window  Help

START CICDtesting ...aaaaabbbbbaaaaabbbbbaaaaaccccccaaaaaccccccaa
testEnd with finalLedCnt=43
```

connect & *.elf
download not shown

Here the **1**.run takes **6**sec (!! random-time !!)
-> finalLedCnt(@250ms)=**22**

-> (finalLedCnt=**22**)%10 = **2**

if (0<= finalLedCnt%10 <5) then char2print='c'

elseif (5<= finalLedCnt%10 <10) then char2print='d'

-> 2.char '**c**'

M3b/mdbcs: Lab for mdbcs -> expected result

Steps to get+run the example for this mod4/`mdbcs` (continued)

10. ... analysis of groovy-script to identify mentioned parts (testrun)

```
by default. If you wish to read custom ranges, please go to the Memories to Program property page and spec
Programming complete


================================================================================
               STOP anytime by pressing SW0...
     ############ Performing test (1.run) -> running 2sec and then halt


Running

Halting...
Target Halted
############ Halted at: 0x166c
############ Address of "finalLedCnt": 0x20000008 Value : 7
############ Address of "char2prntIdx": 0x20000004 Value : 0
############ for 2.run ch2_idx(new): 2



================================================================================
               STOP anytime by pressing SW0...
     ############ Performing test (2.run) -> running 3sec and then halt


Running

Halting...
Target Halted
############ Address of "finalLedCnt": 0x20000008 Value : 18

##### ========================================
##### ======= Success: end of test =======##

======(dev11mod4Lab_mdbcs-run.bat): SUCCESS
#######(dev11mod4Lab_mdbcs-run.bat): DONE

c:\Test\dev11_mod4_mdbcs_test\mu_dev11_mdbcore>
```

```
COM37:115200bps - Tera Term VT
File  Edit  Setup  Control  Window  Help

START CICDtesting ...aaaaabbdddaaaaaddd
testEnd with finalLedCnt=18
```

connect & *.elf
download not shown

Here the **2**.run takes **2sec**
(!! random-time !!) ->
finalLedCnt(@250ms)=**7**

-> (finalLedCnt=**7**)%10 = **7**

if (0<= finalLedCnt%10 <5) then char2print='c'
elseif (5<= finalLedCnt%10 <10) then char2print='d'

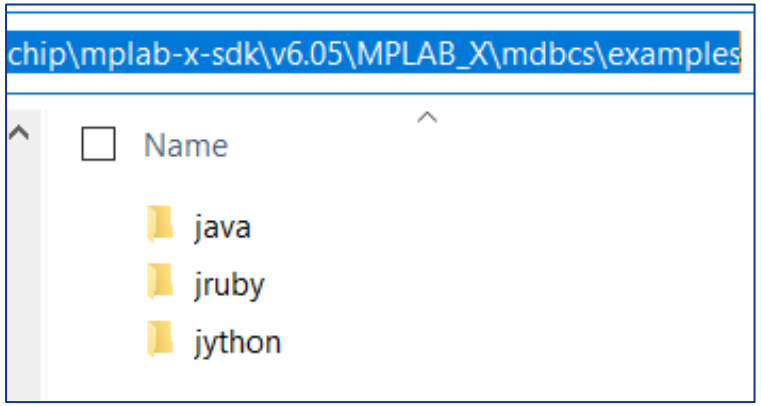
-> 2.char '**d**'

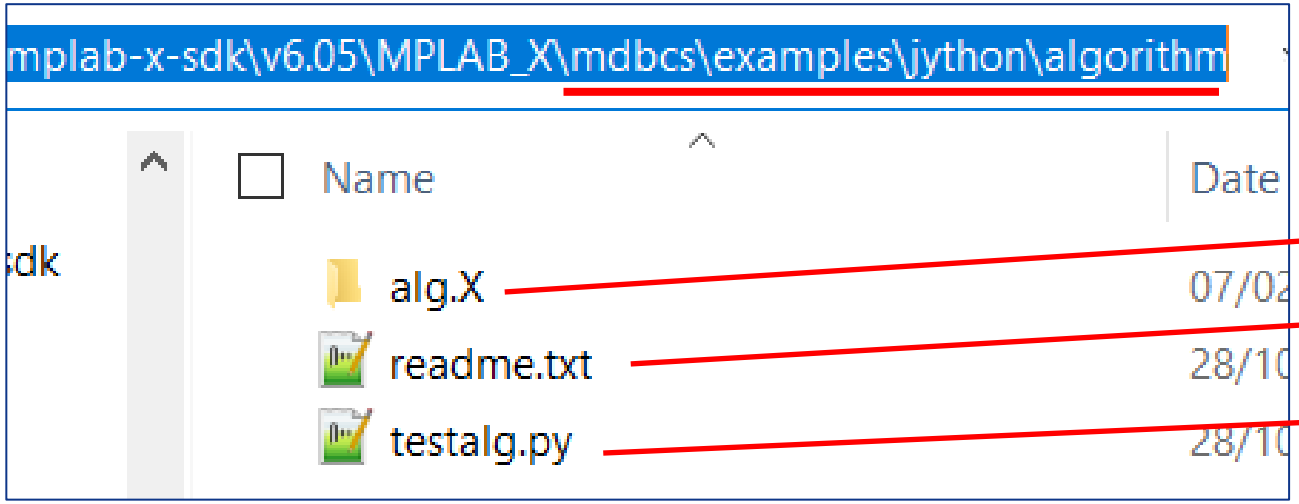
Demo now

M3b/mdbcs: mdbcs examples in SDK

Examples in the SDK

- The SDK holds several interesting examples in different languages Jython, JRuby, Java ***though*** no Groovy which is therefore used in this training
- Examples are inside SDK-installation <sdk>\mdbcs\examples\<language>\
- Each example has a readme.md which explains
  - what the example does
  - how to run it and what HW is needed or simulator is used
- Examples, eg: '***algorithm***' are available as either
  - Jython and JRuby examples are scripts + MPLABX-project





- MPLABX-project
- readme: explains what the example does
- Jython script

M3b/mdbcs: mdbcs - Knowledge Check

Summary and knowledge check module4: using `mdbcs`

- what is `mdbcs` ?

  => wrapper around MDBCore-classes simplifying MDBCore-usage

- how to get and use `mdbcs` ?

  => need to download MPLABX-SDK -> configure env+classpath (config.py??) -> build mdbcs.jar from mdbcs_NB.prj

- features and disvantages or advantages of `mdbcs` ?

  => can evaluate read variables or memory - image good or bad run is defined by building an FFT over a memory-range

  => can use Java-functions like Random() or import more Java-libraries like SwingUI

  => no limits due to scripting-language but requires SDK+configuration+building

  => provides path into Java-world (swingUIs)

  => one disadvantage compared to mdb.bat is that some tools have to be installed and configured

- how to extend `mdbcs` ?

  => use the SDK-docu to find more MDBCore-classes /-functionality

  => SDK provides source-code of `mdbcs`, so you can create your own `mdbcs` version adding any MDBCore function

End Mod3b: mdbcs

End of module3b:

automated FW-testing using MDBCore-IF *mdbcs*

Next module3c:

automated FW-testing using MDBCore-IF *MDBCore*

END

BACKUP

backup slides